

How our sync works

A walkthrough of the remelonDB sync protocol for the NotAnotherCards team

NotAnotherCards

2026-08-07

Why syncing is a real problem

Our app promises something simple to state and hard to build: review your flashcards anywhere, with or without internet, on your phone or in the browser, and everything ends up consistent. You rate twenty cards on the train with no signal. At home, your laptop already shows the results.

Offline-first, multi-device is less a feature of the app than its architecture. Sync is what makes the local database we have already built more than a per-device data store. Anki is the reference point here: its sync is the feature users rely on most, because flashcard usage naturally splits across contexts, review on the phone, manage on the laptop. Without sync we would either go online-only, losing the offline use case and most of the point of the local database, or stay single-device, which would be a much weaker product.

The hard part is that devices can change things while they are out of touch with the server. Your phone edits a card on the train; your laptop edited the same card yesterday evening, and the phone has not synced since before that. Nothing happened at the same time, yet when the phone comes back online there are two versions of one card. Something has to decide what the truth is now, and every device has to converge on it, without losing anybody's work and without quietly bringing back things that were deleted. Naive implementations tend to fail in one of two ways, and both failures are silent: a change gets lost, or a deletion comes back.

The rules for how a device and the server talk are written down in one normative document, `sync-wire.md` in the remelonDB repository. It is a contract in the RFC sense (MUST/SHOULD/MAY), the client implements one side, every backend implements the other, and a conformance suite tests compliance mechanically. This walkthrough covers the same ground in prose, at a level where you could review a sync-related PR without reading the engine source first. If you take away one sentence, take this one: **a device never overwrites the server blindly, and the server never refuses anything silently.**

Two operations, nothing else

The protocol consists of exactly two request types.

Pull: “give me everything that changed since position X in your history.” The server answers with the changes and a new cursor. A brand-new device pulls with `cursor: null` and receives the complete current state.

Push: “here are my local changes, made on top of position X.” The device sends its created records, updated records, and deleted ids; the server applies them atomically and answers.

A full sync is pull-then-push, and `synchronize()` in the client drives that loop. There is no third request type, and the transport is pluggable: our API binds the two operations as `POST /sync/pull` and `POST /sync/push`, every protocol outcome carried in a 200 body, while transport-level failures (401, 5xx) surface as a failed sync with local state untouched.

The wire shapes are small enough to quote in full. A pull request and its two possible answers:

```
{ "cursor": Cursor | null, "schemaVersion": number, "migration": {...} | null }
```

```
{ "changes": Changes, "cursor": Cursor }    // normal
{ "resyncRequired": true }                  // cursor expired/unknown
```

The three request fields cover the three ways a device can be behind. `cursor` says where it left off in the server’s history; `null` means “I have nothing, send the complete state” (fresh install, new login, or a rebuild after a resync). `schemaVersion` says which version of the table schema the client runs, so the server can notice a client ahead of or behind what it expects instead of mis-parsing rows. `migration` is `null` in the common case and becomes non-null exactly once after an app update that added synced tables or columns: it lists what is new since the last-synced version, and the server then includes full current records for those tables and columns regardless of the cursor. A device can be perfectly up to date by cursor position and still never have received data for a column that did not exist in its old schema; this field closes that gap. In short: behind in time (cursor), behind in shape (schemaVersion), freshly reshaped (migration).

A push request and its two possible answers:

```
{ "changes": Changes, "cursor": Cursor }
```

```
{ "cursor": Cursor | null, "changes": Changes | null,
  "rejected": { [table]: RecordId[] } }    // accepted
{ "conflict": true }                      // stale push
```

`Changes` is a per-table object of `{ created: Record[], updated: Record[], deleted: RecordId[] }`. Records travel whole, all user columns plus `id`; the client’s internal bookkeeping fields (`_status`, `_changed`) never appear on the wire in either direction, and unknown keys are stripped by the client rather than treated as errors. One deliberate looseness: strict created/updated classification is not required. The client applies an `updated` for a locally missing record as a create (and vice versa), and a server may report all live rows as `updated`. If a changeset names one id in both arrays, the last statement wins; it must not fail.

The cursor, and why it is not a timestamp

The cursor deserves its own section because the whole protocol rests on it. To the client it is an opaque string: store it, echo it back, never inspect or compare it. To the server it identifies an exact position in the commit history of that user’s data.

Our backend implements it as a revision sequence: a global counter, and every committed change stamps its rows with the next value. A pull “since cursor 41” is then a query for rows with revision greater than 41, served from one consistent snapshot, and the returned cursor names that snapshot. This is the invariant everything else leans on, the *snapshot rule*: every change committed after the

pull’s snapshot, including changes that were in flight concurrently and committed a moment later, must be returned by some future pull from that cursor. Nothing is ever allowed to fall between two pulls.

That last clause is why timestamp-based sync (“give me rows where `last_modified > 14:32:07`”) is non-conforming, not merely unfashionable. Wall clocks on two machines disagree; more subtly, two transactions can *write* their timestamps in one order and *commit* in the other. A pull that lands between the two commits reads the later timestamp but not the earlier row, and that row is now invisible to every future incremental pull. No error is raised; a card is simply missing on one device forever. Commit-ordered cursors close this gap by construction, and the conformance suite includes a concurrency scenario (a write committing during a pull) that timestamp implementations reliably fail. On the Postgres side, keeping the commit path serialized per user is what the per-scope advisory lock in the store is for.

None of this bans timestamps from the app. Our rows are full of them: `created_at` orders deck lists, `due_at` drives the review scheduler, `reviewed_at` is the review history. Those are data, owned by the device that wrote them, and the sync layer carries them without ever reading them. The rule is only about the delivery mechanism: what a device has or has not seen is tracked by revision, never by comparing clocks. Two clocks, two jobs: row timestamps say what happened when in the user’s world; the cursor says who has been told what.

Deletions are the hard part

A deletion cannot travel as an absence. If the server simply removed the row, a device that was offline during the deletion would still have the record, see nothing about it in any pull, and happily push it back at the next opportunity. Deleted data would resurrect constantly, which is both a bug and a privacy problem.

So deletions are first-class: the wire carries them as ids in `deleted`, and the server keeps a *tombstone* per deleted id, a row that remembers “this id died at revision N” while its user content is scrubbed immediately. Tombstones let every device, however late it syncs, learn about the deletion through an ordinary incremental pull. They also guard the write path: a push that tries to write to a tombstoned id is refused by the store, and, by contract, that refusal must be reported (more under *Rejections* below). Content is scrubbed the moment the deletion commits, which matters for data protection.

When does a sync run?

The protocol has no opinion: `synchronize()` is a function, and calling it is app policy. Everything about the design makes calling it safe at any moment: apply is idempotent, a conflicted push retries in a bounded loop, and a failed transport leaves local state untouched. The usual triggers are app start, regaining connectivity, returning to the foreground, and a debounce after a burst of local writes, with a periodic timer as backstop.

This is something we still have to decide and implement, for web and mobile. Until then, everything in this document still holds, the local database simply accumulates dirty records and the first sync carries them.

Inside the client

The server never sees it, but the client’s half of the contract rests on two hidden columns every local record carries: `_status` and `_changed`. `_status` is the record’s sync lifecycle state: `synced` (clean), `created` (new since last sync), `updated` (edited since last sync), or `deleted` (marked for deletion, hidden from queries, awaiting push). `_changed` lists which columns were touched while dirty, and that list is what makes per-column conflict merging possible: after a conflict pull, exactly the columns in `_changed` keep their local values.

`synchronize()` reads the wire’s `Changes` and applies them with one discipline throughout: apply is idempotent and equality-gated. Re-delivering a change the client already has is absorbed by comparing values, never an error. That single property is what lets the rest of the protocol stay simple; the degraded push mode, resync replacement, and plain retries all lean on “applying twice is harmless”. Dirty records are protected during apply: a server change to a record you have edited locally does not stomp your unsaved columns, it merges around them.

Collecting outgoing changes is equally mechanical: everything with `_status` of `created` or `updated` goes whole into the push (all user columns, not a diff, which keeps merge logic on the client and the server simple), and `deleted` records contribute only their ids. After an accepted push, everything not named in `rejected` flips to `synced` and `_changed` empties.

The obligations run both ways: the server may rely on the cursor coming back byte-identical, on idempotent apply, on records arriving whole, on the internal columns never reaching the wire, and on rejected records staying dirty and retrying. Anyone writing a second client against the same server inherits that list.

What app code actually sees: the Q DSL and reactive queries

Nothing in this document so far is what feature code touches. Components never call pull or push; they read and write the local database, and the sync layer works underneath. The reading side goes through the query DSL, Q:

```
const dueCards = db.get(UserCard).query(
  Q.where('due_at', Q.lte(now)),
  Q.sortBy('due_at', Q.asc),
)
const cards = await dueCards.fetch()           // one-shot
const stop = dueCards.observe(render)         // reactive
```

Joins and counts follow the same shape. The cards of one deck, newest first, and a live count of everything due:

```
const deckCards = db.get(UserCard).query(
  Q.where('deck_id', deck.id),
  Q.sortBy('created_at', Q.desc),
)
dueCards.observeCount(setBadge) // re-fires as the number changes
```

The difference between the two calls is the heart of the design. `fetch()` asks a question once, and the answer is stale the moment anything changes. `observe()` subscribes to the question: the callback receives the results now, and again every time they change, for any reason: the user edits a card, a review event lands, or a background sync applies a pull from the server. A component

simply renders whatever its observation last delivered; that is all “reactive” means. The payoff shows in the sync case: when `synchronize()` applies the laptop’s changes, the phone’s observers fire and the UI updates, with no component knowing sync exists.

This is why queries are data, not SQL strings: `Q.where`, `Q.sortBy`, `Q.on` for joins compose into a plain structure the engine can read, so it knows which changes affect which query and which observers to wake, something no SQL string offers. Being data also means the same query runs unchanged on the web driver, the React Native driver, and the node driver our tests use, which is what keeps `@repo/offline-db` a single shared package instead of three per-platform dialects. The engine appends its own conditions too: every query silently excludes records whose `_status` is `deleted`, which is why app queries never filter on deletion themselves and why a `markAsDeleted()` record vanishes from every list immediately, long before any server round-trip. On the web, the React bridge (`useQuery`, `useDatabaseState`) wraps observation into hooks; the mobile side uses the same pattern against the same queries.

Writes mirror reads. Everything happens in a `db.write()` transaction, applies locally and instantly, marks the records dirty, and returns; the dirty records ride out on the next push:

```
await db.write(async () => {
  const card = await db.get(UserCard).create({
    deck_id: deck.id, front, back,
    due_at: now, created_at: now, updated_at: now,
  })
  await staleCard.update((r) => { r.back = fixed; r.updated_at = now })
  await oldCard.markAsDeleted()
})
```

In React, the bridge hooks wrap query observation so a component is one line away from live data:

```
const { data: decks } = useQuery(() => (db ? getDecksQuery(db) : null), [db])
```

The UI never waits for the network. One habit follows: components read the database, not push responses. Whether a write has synced is the database manager’s business (`useDatabaseState` exposes it), not something feature code should track by hand.

Conflict: when both sides changed the same record

A push carries the cursor of the pull it was based on, which lets the server detect staleness precisely: if any pushed record was modified on the server after that cursor, the entire push is answered with `conflict` and nothing is applied. Pushes are atomic, all accepted records commit together or none do, so a half-applied push cannot exist.

Resolution happens on the client, where both versions are available. It pulls, merges per column (columns with local unsaved edits keep the local value, everything else adopts the server’s), and pushes again. The loop is bounded, five rounds by default, after which the app surfaces a sync error instead of spinning. Per-column merging is deliberately the client’s job, not the server’s: the server would have to guess, while the client knows exactly which columns the user touched.

The practical consequence: nobody’s edit disappears without trace, and whole-push conflict keeps the server’s reasoning trivial while the client recovers the fine granularity during its merge.

Rejections: when the server says no to a single record

Conflict is about timing; rejection is about content. A push can contain a record the server will not accept regardless of timing, one that fails validation, references another user's data, or targets a tombstoned id. Rejection is per record: the server applies the acceptable rest of the push and names the refused ids in **rejected**, grouped by table. The client keeps those records dirty, so they are visible as unsaved and will retry on the next push rather than vanish.

The contract states the underlying principle in bold: **refusals are never silent**. Everything the server declines to apply must be visible in the response, either as **conflict** or as an id in **rejected**. The nasty failure mode this outlaws: a server that quietly drops a write while reporting success leaves that device convinced its data is saved. Its cursor moves past its own push, so no future pull ever corrects the record, and the device has diverged permanently, with zero errors anywhere. The conformance suite pins this rule with dedicated cases, including the tombstone variant.

For feature work this yields a simple habit: when a write matters, read the push response. The protocol guarantees the server told you; the app still has to look.

The interleave: what a push answer carries

A successful push answers with more than “ok”. While the pushing device was offline, other devices may have committed changes; the push response carries exactly those *foreign* changes, everything committed between the request cursor and the push's own snapshot, excluding the push's own records (a device never receives its own writes back, so apply stays cheap). Cursor and interleave are a package: returning a new cursor without the complete interleave would skip those foreign changes forever, the same lost-write hole the snapshot rule closes on the pull side.

Computing the interleave completely is not always possible, and the protocol makes the honest escape explicit: a server may answer **cursor: null, changes: null**, “applied, but no interleave”, and the client picks everything up on the next pull, its own echo absorbed by idempotent apply. This *degraded mode* is mandatory when the request cursor is older than the server's tombstone retention (next section): a “complete as far as I know” interleave would silently resurrect a deleted record.

Retention, GC, and the emergency exit

Tombstones and change history cannot be kept forever. The server prunes old tombstones periodically; the *retention floor* is the oldest revision it can still serve completely. A cursor older than the floor cannot be answered honestly, because the deletions between that cursor and the floor are gone; serving partial data would drop deletions silently. The only lawful answer is **resyncRequired**.

On receiving it, the client rebuilds: it pulls from **null**, reconciles against the full snapshot with replacement semantics, destroys local synced records that no longer exist on the server, and keeps local dirty records, which merge and push as usual. Slow and safe over fast and wrong. From the app's perspective the whole mechanism reduces to one rare, well-defined event: a device away too long starts fresh and loses nothing it authored.

Schema changes ride the same machinery: after a local migration the pull request carries a **migration** block (last synced version, new tables, new columns), and the server responds with full current records for the affected tables regardless of the cursor, because the client just gained

schema it has never synced. This is why changing a synced table is a coordinated step and not a quick local edit.

One sync, played through

Monday evening, phone and server agree; the phone holds cursor 41. Tuesday on the train, no signal: you review twenty cards (twenty append-only review events, twenty cards with new due dates) and fix a typo in one card's answer. All of it is recorded locally and marked dirty. Monday night, meanwhile, your laptop had edited that same card's answer and synced it; the server committed it as revision 42.

Back in signal, the phone syncs. **Pull** with cursor 41: the server returns the laptop's edit (revision 42) and cursor 42. The phone merges per column; the answer column is locally dirty, so your train version of that column survives, while any column the laptop alone touched adopts the laptop's value. **Push** with cursor 42: forty-one records travel; nothing on the server changed after 42, so there is no conflict. The server validates each record, applies all of them in one transaction, stamps them revision 43, and answers with cursor 43 and an empty interleaved. The phone marks everything synced. Two requests total.

The variations (a conflicting laptop sync in between, a malformed event in the batch, a phone left in a drawer past retention) each land in one of the paths above: conflict, rejection, or resync. Every path converges, with the train reviews intact.

How the server half is assembled

Knowing where the moving parts live makes sync PRs much easier to review. The backend is three layers, each with a narrow job.

The **engine** (`@remelondb/server`) implements the protocol semantics once: cursor decoding, the snapshot rule, conflict detection, the rejection bookkeeping, the interleaved computation, degraded mode. It is the layer this whole document describes, and it is app-agnostic; it knows tables only by name.

Below it sits the **store seam**: nine small methods (`changedSince`, `upsert`, `tombstone`, `tombstonedIds`, and friends) that know rows, revisions, and scopes, and nothing about cursors or conflicts. Our implementation is `@remelondb/store-drizzle` over Postgres: a config block per synced table pointing at the machinery columns, with the protocol obligations earned by a global revision sequence and a per-user advisory lock serializing the commit path, exactly the mechanism the cursor section demanded. An in-memory store ships as reference and test double; the conformance suite runs against both.

On top, the **NestJS binding** exposes `POST /sync/pull` and `POST /sync/push`, stamps the sync scope from the authenticated session (which is why client rows carry no `user_id`), and hands the request to the engine.

NAC's own code enters through three engine hooks, and this is where sync-related app PRs usually live. Per-table `validate` runs each incoming record through the shared Zod row schema. `appendOnly: true` on a table makes the engine reject writes to existing ids, which is how review events are enforced as history. And `crossValidate` is the app-level pass for relational rules the engine cannot know: cards must reference a live deck owned by the same user, deleting a deck cascades to its cards and their review events, and so on. Everything rejected by any of these lands

in the same **rejected** list the client already knows how to handle, so app policy and protocol policy speak one language.

How we know it works

A contract is worth what its enforcement is worth. This one is checked three ways, all automated.

First, the protocol design exists as a small formal model (Quint), whose invariants are checked when the model changes. This is design-level verification: it explores interleavings of pulls, pushes, GC, and crashes that are impractical to enumerate by hand.

Second, the spec ends with a ten-item conformance checklist that ships as a runnable vitest suite (`@remelondb/server/conformance`), and our NestJS/Postgres backend runs it in CI against a real database on every relevant push. The items are concrete scenarios, from “full pull is complete and scoped to the caller” and “a change committing during a pull is never lost” through “a stale push conflicts and applies nothing” to “nothing of another user’s data ever crosses”. When we extend the server, this list stays green or the change does not merge.

Third, a documented tour of request/response pairs replays against the reference server in CI, so the prose documentation cannot drift from the implementation.

The short version: the sync rules are a written contract, the server is tested against them mechanically, and every scenario in this document corresponds to a test someone can run.

Day-to-day rules that follow

- **Delete through the API, not by flag columns.** `markAsDeleted()` is what lands a record in the wire’s **deleted** list and produces a server tombstone. A hand-rolled **deleted_at** column is, to the sync layer, an ordinary update: the “deletion” will not propagate, and after a sync round-trip it can resurrect. This is also why the client rows carry no **user_id** or **deleted_at**: scoping is stamped server-side from the session, deletion is protocol state, and neither belongs to app data.
- **Review events are append-only.** A rating is history; correcting yourself means rating again, not editing the event. The server rejects updates to review events (an engine-level table policy), and the app sees that honestly in **rejected**.
- **Read the push response where writes matter.** Rejection is per record and guaranteed to be reported; the UI layer decides what to do with it, but ignoring it forfeits the guarantee.
- **Schema changes to synced tables are coordinated changes.** The migration mechanism handles them, but it must be driven: bump the schema version, ship client and server in an agreed order, and expect the first pull after migration to be larger than usual.
- **Timestamps on synced rows are client-owned epoch milliseconds.** The server must not default or auto-update them: the client’s values are the data, and the sync layer orders by revision, not by time.

Debugging sync: symptom to cause

When sync misbehaves during development, the protocol’s guarantees make the differential diagnosis short. These are the patterns worth knowing before staring at logs.

A deleted record keeps coming back. Almost always a deletion that never became protocol state: something wrote a flag column instead of calling `markAsDeleted()`, so the wire shipped an

update, and the server happily upserted the record alive again. Check how the delete is performed before suspecting the server.

A device shows stale data but no errors. First question: is it actually pulling? A stored sync error, a dead adapter, or an authentication failure surfaces as thrown transport errors, and those leave local state untouched by design. If pulls succeed and data is still missing, that would be a snapshot-rule violation, which is conformance-suite territory; check whether the backend passed the concurrency scenario before assuming one.

Sync loops and eventually reports failure. The bounded conflict loop ran out of rounds, which means something is committing to the same records between every pull and push. Usually another device in a tight sync loop, a background job editing synced rows server-side, or a test writing to the database mid-sync. The fix is finding the concurrent writer, not raising the round limit.

A write seems saved but never reaches other devices. Read the push response of the writing device: the id is almost certainly in `rejected`, and the record still dirty locally. Then ask why the server refused it: validation, ownership, append-only policy, or a tombstoned target. If the response claims success and the record still diverges, that would be a silent-refusal bug, which the contract forbids and the conformance cases now pin; report it against the library, with the response body attached.

Local tests pass, real sync misbehaves. Check the test store: the in-memory reference store and the Drizzle store both pass conformance, but app-level hooks (`validate`, `crossValidate`, `appendOnly`) only run where they are configured. A test that talks to the store directly bypasses the engine, and with it every protocol rule this document describes; meaningful sync tests go through the engine's handlers.

The rule underneath: the protocol never fails silently, so start every diagnosis by reading what the responses actually said. The answer is usually in the envelope before it is in the logs.

Glossary

Term	Meaning
Cursor	Opaque token naming a position in the server's commit history; the client stores and echoes it, never interprets it.
Tombstone	The server's record that an id was deleted (content scrubbed immediately), kept so late devices learn of deletions.
Conflict	Whole-push answer when any pushed record changed on the server after the push's base cursor; nothing applies.
Rejected	Per-record refusal list; refused records stay dirty on the client and retry. Refusals are never silent.
Retention floor	Oldest revision the server can still serve completely; older cursors get <code>resyncRequired</code> .
Resync	Full rebuild from a <code>null</code> pull with replacement semantics; local dirty records survive.